جامعة الملك عبدالله للعلوم والتقنية
King Abdullah University of Science and Technology

أكاديمية كاوست
KAUST ACADEMY

# Internet of Things (IoT)

**DAY 4** Out to the internet: MQTT, the cloud, and chat control

August 2 to 6, 2026

KGSP Summer Enrichment Program 2026
KAUST Academy, KAUST

Salah Abdeljabar
salah.abdeljabar@kaust.edu.sa

# 🗺️ On Today's Agenda

1. **Telemetry**: what to send, how often, and what if the link drops
2. 🔧 **Project 9**: live temperature and humidity on a page that updates itself
3. **MQTT**: the broker, topics, wildcards, and getting a message delivered; brokers, tooling, and a Python client to try it yourself; MQTT 5.0's new features
4. 🔧 **Project 11**: publish to a cloud dashboard, and be commanded back
5. **The reference architecture**: the map, now that you have built one
6. **Zooming out**: the general pattern behind any device cloud, no-code automation, and dashboards
7. 🔧 **Project 12**: read the sensor and switch the light by chat, from anywhere
8. **Security**: who is allowed to connect, and what your token is worth
9. 🎓 **The capstone brief**: form pairs, sketch an idea

By the end your board is reachable from anywhere, not just from this room.

# Telemetry 

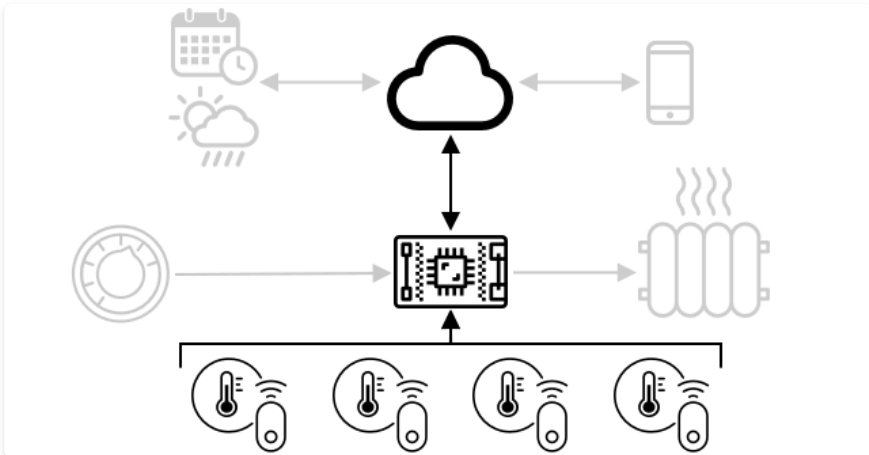
Sending sensor data from device to cloud

# What is Telemetry?

From the Greek for *"measuring remotely"*: gathering sensor data and sending it to the cloud, usually as **JSON**.

```
{ "light": 300, "temperature": 22.5 }
```

The first telemetry device (1874) sent Mont Blanc weather to Paris over wires.



Readings from the room's sensors are packaged up and sent to the cloud as telemetry.

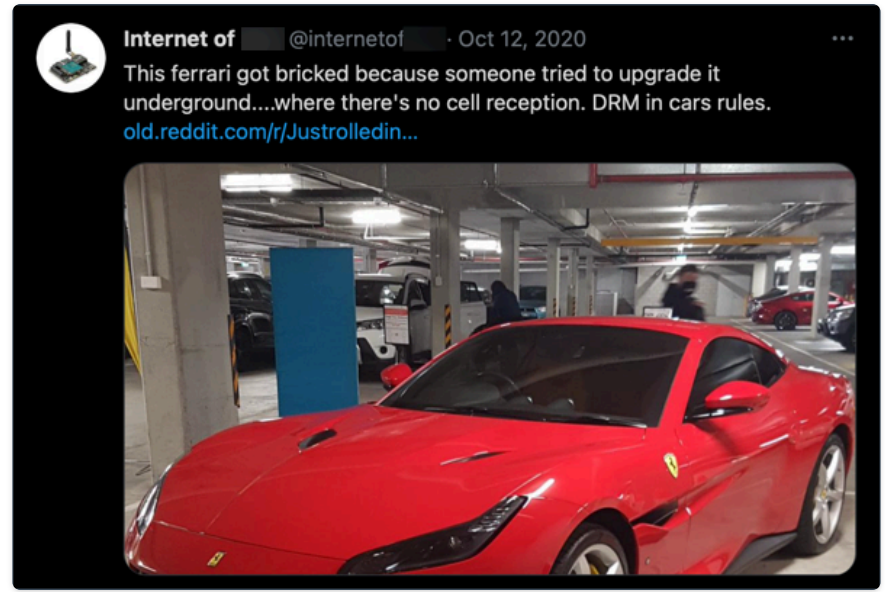
# Frequency & Connectivity

**How often to send?**

**Too often** → wastes power, bandwidth, and money. **Too rarely** → you miss events. Match the rate to the job.

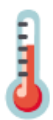
**Caution**

Connections drop. Smart devices should still work **offline**: a thermostat keeps deciding locally.




No signal, no cloud.

# The DHT11: Humidity & Temperature

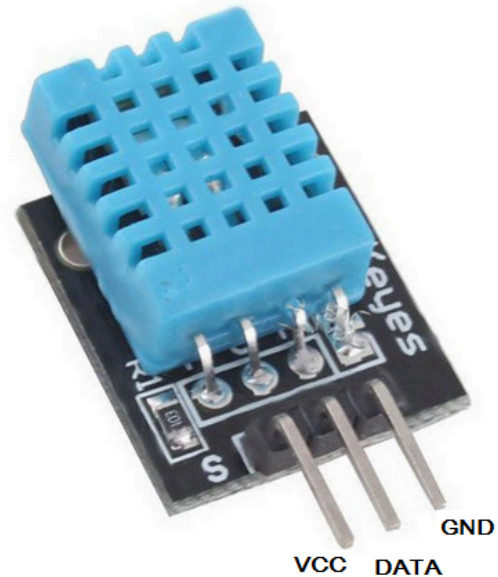


A single digital sensor, two measurements, one wire

# Two Sensors, One Package

Pre-calibrated, so it needs **no external circuit**: an 8-bit microcontroller inside converts raw readings straight to finished values.

- **Humidity**: a resistive element between two electrodes. Absorbed moisture releases ions, lowering resistance as humidity rises
- **Temperature**: an **NTC thermistor**, resistance drops as it gets hotter
- Range: **20-90% RH**, **0-50°C**, accuracy **±1% RH** / **±1°C**
- Sampling rate: about **1 Hz** (once a second)




DHT11 Sensor Module

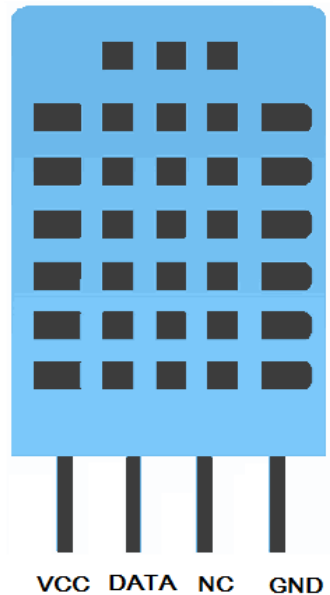
Source: engineersgarage.com

# Pinout: Bare Sensor vs Module

The bare part has **4 pins**; the **3-pin module** on the previous slide drops the unused one and adds the pull-up for you.

- **VCC**: 3.3-5.5 V
- **DATA**: one wire, both directions. Needs a **10 kΩ pull-up** to VCC. On a bare sensor, that's an external resistor; the module builds it in
- **NC**: not connected
- **GND**

> **Note**
>
> At 5 V the DATA line can run up to 20 m; at 3.3 V, closer to 1 m.



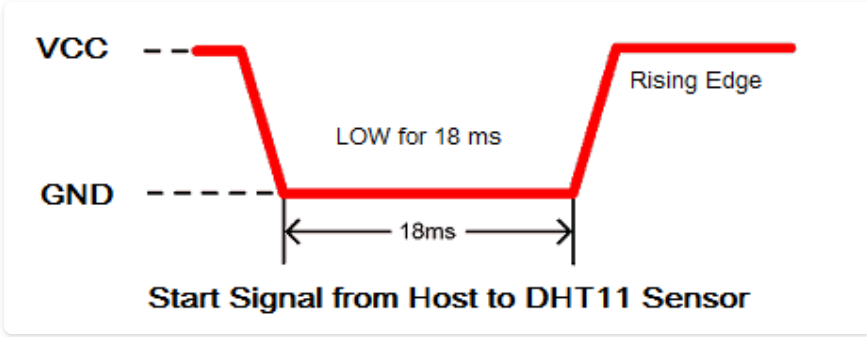

DHT-11 Sensor Pinout

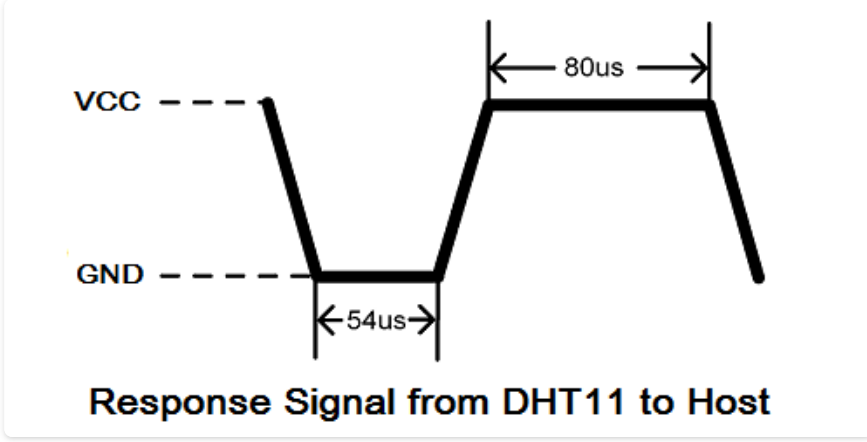
Source: engineersgarage.com

# A One-Wire Conversation

The library's `read()` call hides a four-step exchange on that single DATA wire.

- **1. Start signal**: the host pulls DATA **LOW for 18 ms**, then releases it (rising edge)
- **2. Response**: the DHT11 answers with **LOW for 54 µs**, then **HIGH for 80 µs**, confirming it's ready to send data
- Steps 3 and 4 (the data bits and end of frame) are next



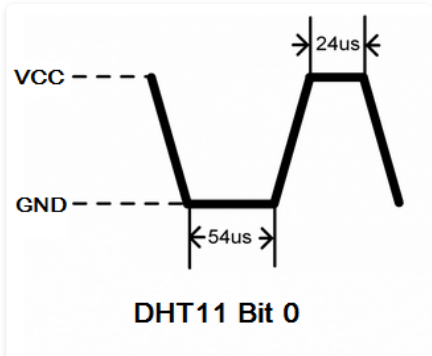



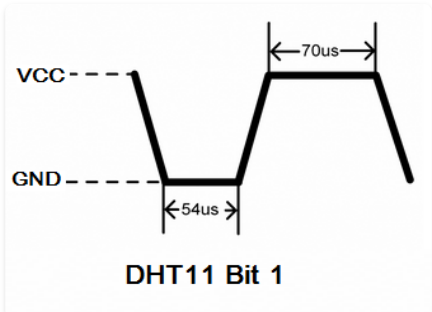

Source: engineersgarage.com

# Encoding Bits by Pulse Width

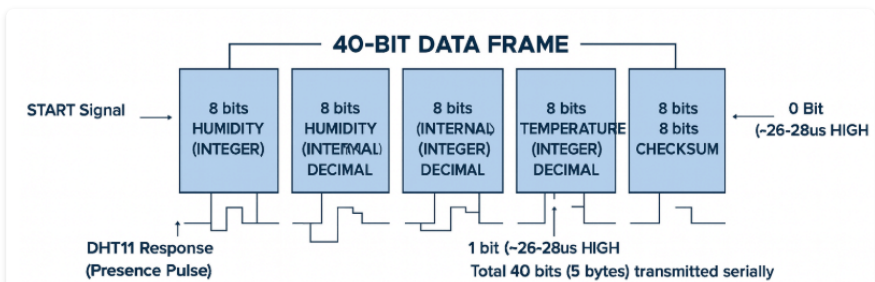
**3. Data**: 40 bits, sent as pulses. Every bit starts with the **same 54 µs LOW**; only the **HIGH that follows** tells 0 from 1.

- **Bit 0**: HIGH for **24 µs**
- **Bit 1**: HIGH for **70 µs**
- 40 bits = 5 bytes: **humidity** (2), **temperature** (2), **checksum** (1, the binary sum of the other four)
- **4. End of frame**: DATA drops LOW once more, then the sensor sleeps until the next start signal




Same LOW, different HIGH: that's the whole encoding scheme.




Source: engineersgarage.com, digi-electronics.com

Live preview, view online at the workshop site

2026-iot.abdeljabar.net

# DHT11 Web Server 🌡️

Live temperature and humidity on a web page that updates itself.

- Read a real environmental sensor through a library
- Handle a failed reading with `isnan()` instead of a nonsense number
- Fill page placeholders with live values via a template processor
- Compare a digital sensor’s numbers with Project 2’s raw ADC voltage

Key pin: DHT11 `4`

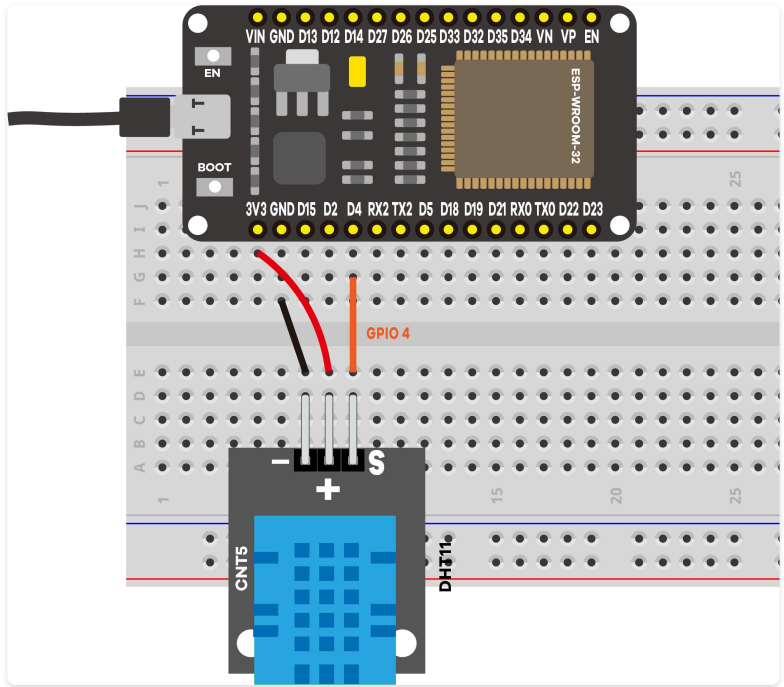
Open the guide ↗

# A Digital Sensor

The **DHT11** reports **temperature (°C)** and **relative humidity (%)** over a single data wire.

- **S** (signal) → **GPIO 4**, **+** → **3.3 V**, **–** → **GND**
- A 3-pin module with the pull-up built in, so no extra resistor

> **Note**
>
> Contrast Project 2: an **analog** sensor gives a voltage you must interpret; a **digital** sensor hands you finished numbers.



# The Library Does the Hard Part

```
DHT dht(DHTPIN, DHTTYPE);          // pin + type + protocol code, bundled

float t = dht.readTemperature();   // degrees Celsius, already converted
if (isnan(t)) return "--";         // failed read: show a dash, not garbage
return String(t);                  // text, because it goes into a web page
```

- The DHT11 speaks a one-wire **pulse protocol**; `dht.readTemperature()` hides the microsecond timing you would otherwise write by hand.
- `isnan()` catches a failed reading. The library returns `NaN`, which is not equal to anything, so it needs its own test.
- A DHT11 fails now and then (a loose wire, or being polled too fast), so a fallback matters.

# Never Blank, Never Stale

The first page load is filled by a **template processor**; after that the browser **polls** each value every 10 seconds.

```
String processor(const String& var){
  if (var == "TEMPERATURE") return readDHTTemperature(); // fills %TEMPERATURE%
  if (var == "HUMIDITY")    return readDHTHumidity();
  return String();
}
```

```
setInterval(function () {                    // every 10 s
  xhttp.open("GET", "/temperature", true);   // innerHTML swaps just the number
}, 10000);
```

- Placeholders make the first load carry real values; polling keeps them fresh.
- **10 s** matches the DHT11's roughly one-reading-per-second rate; polling harder gains nothing.

# A Live Dashboard

The readings update on their own. Breathe near the sensor and the humidity climbs.

**What you learned:**

- A digital sensor hands you finished numbers; a **library** hides a fiddly protocol
- Always check for a failed read: a dash beats a wrong number
- Sensors have a **sampling rate**; match your refresh to it
- Project 11 sends the same readings to the **cloud**, to watch from outside the building

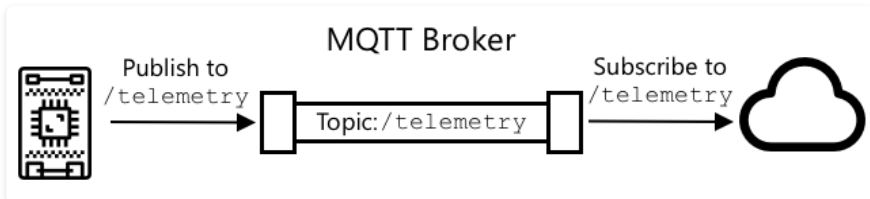
# MQTT 

The standard messaging protocol for IoT

# One Broker, Many Clients

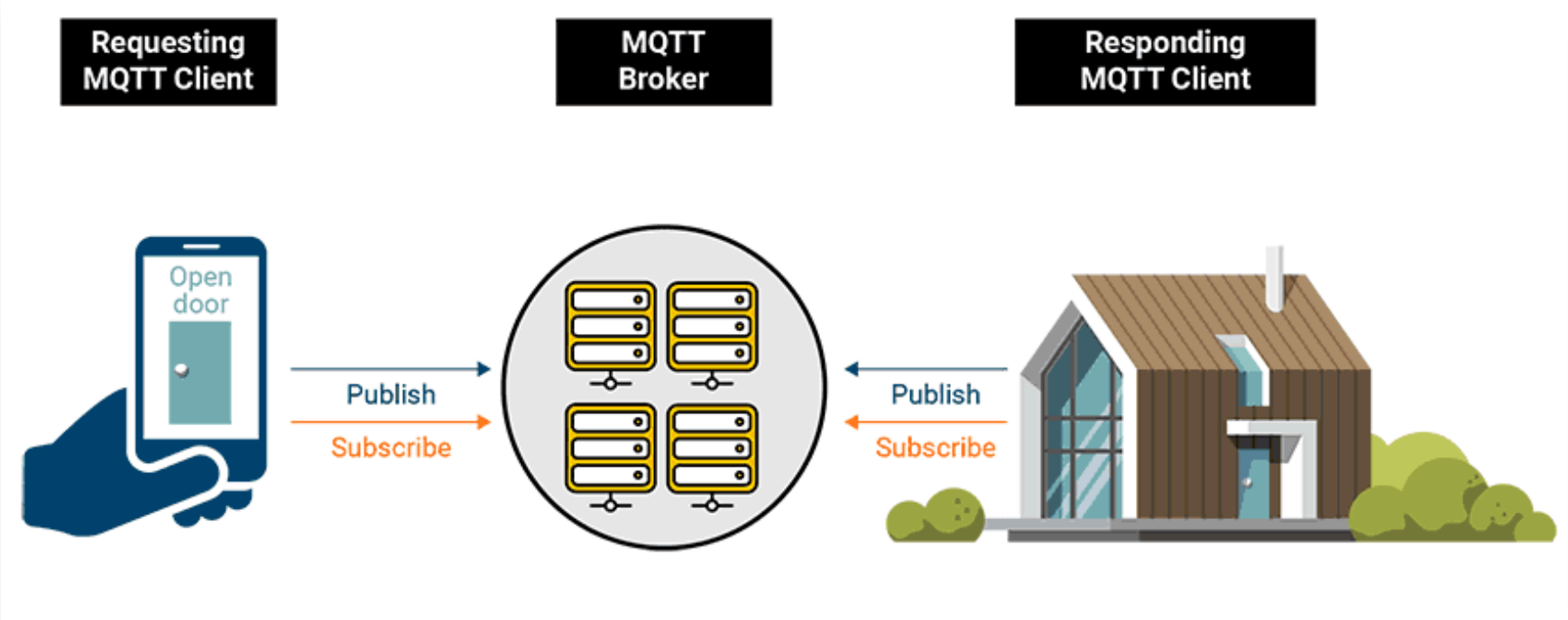
**MQTT** (**Message Queuing Telemetry Transport**) was designed in **1999** to monitor oil pipelines, later open-sourced by IBM.

- All clients connect to a single broker
- Messages route by named topics, not to clients directly
- Topics are hierarchical: `/device-id/telemetry`
- Wildcards: subscribe to `/telemetry/*`




Every client connects only to the broker; topics decide who receives what.

# 🔄 Publish / Subscribe in Action




Source: HiveMQ: MQTT Essentials Part 1

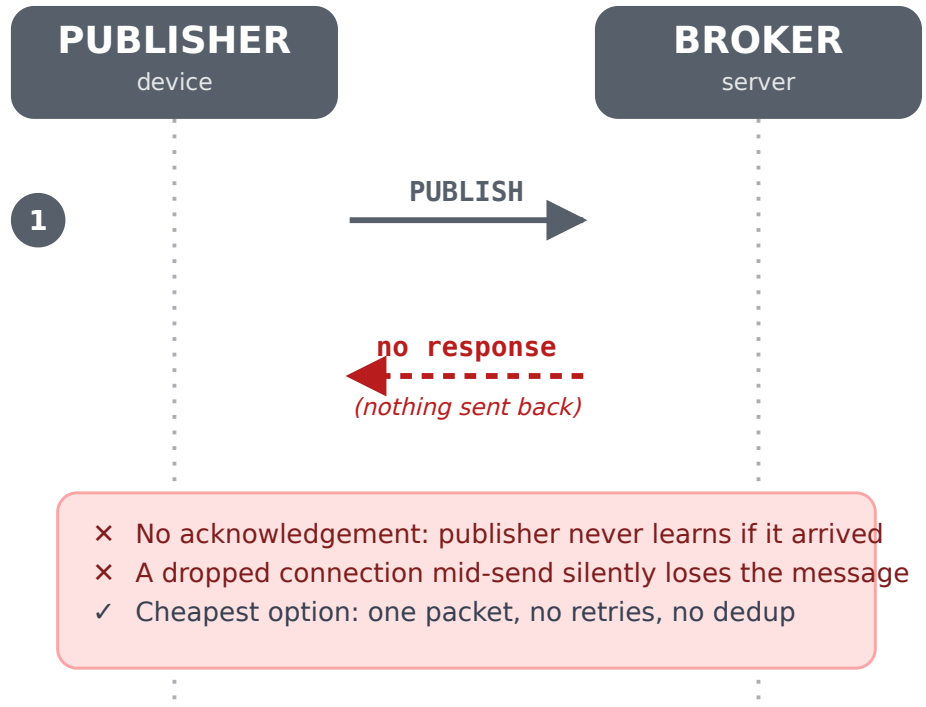
# Quality of Service (QoS) 🤝

MQTT’s core reliability knob: how hard the protocol tries to get a message there

# QoS 0: At Most Once

The publisher sends **one** `PUBLISH` packet and moves on. No acknowledgement comes back, and no record of the message is kept for a retry.

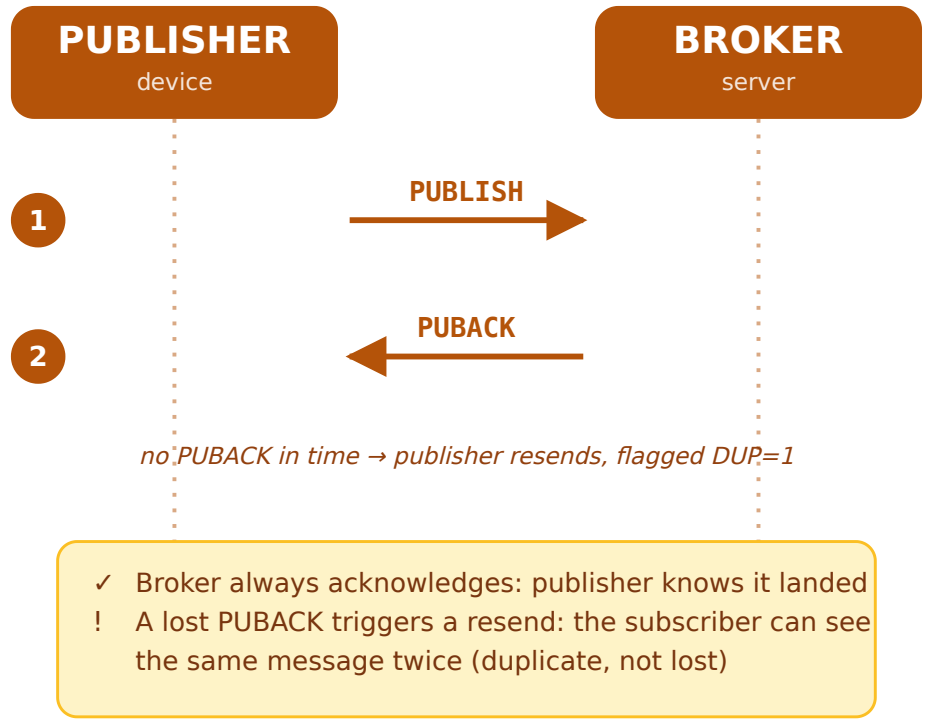
- **Guarantee**: none. Delivery is "best effort" only
- **Cost**: cheapest possible, a single packet, no state to track
- **Risk**: a dropped connection mid-send loses the message silently
- **Use it for**: high-frequency, low-stakes readings where the *next* value arriving is more useful than replaying a missed one (e.g. a sensor publishing every second)



# QoS 1: At Least Once

The broker sends a `PUBACK` back for every `PUBLISH`. If the publisher doesn't see that acknowledgement in time, it **resends** the same packet, marked with a `DUP` flag.

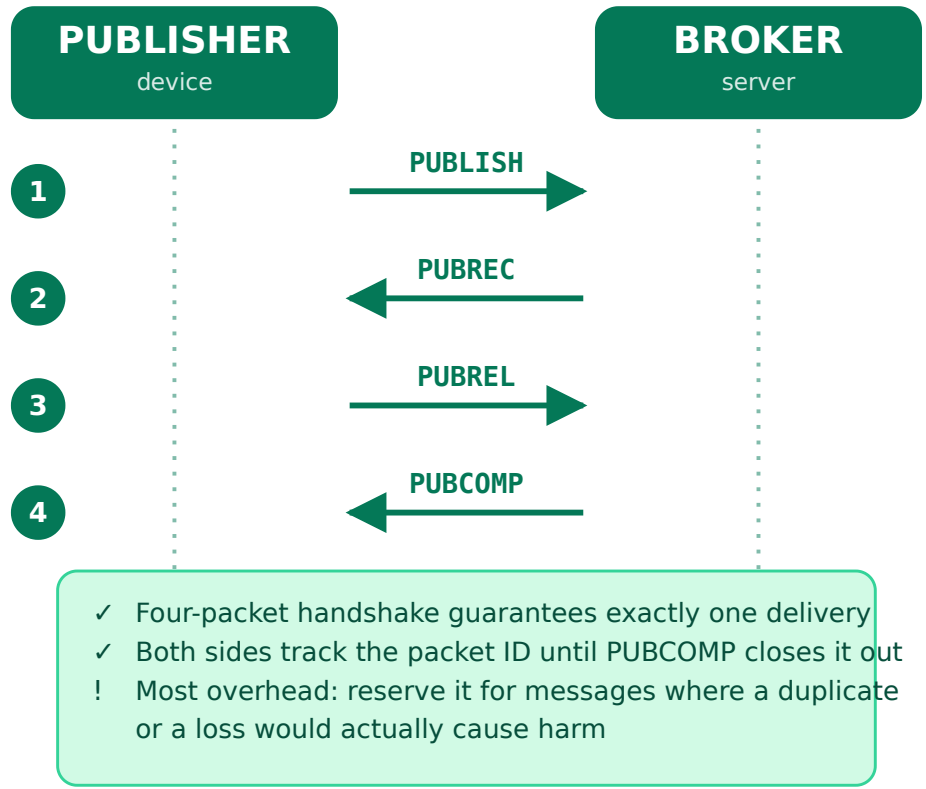
- **Guarantee**: the message *will* arrive, at least once
- **Cost**: one extra packet (`PUBACK`) per message
- **Risk**: a lost `PUBACK` (not a lost message) triggers a resend, so a subscriber can see the **same message twice**
- **Use it for**: readings and events where losing one silently is worse than occasionally double-processing one, e.g. a device status update



# QoS 2: Exactly Once

A **four-packet handshake**: `PUBLISH` → `PUBREC` → `PUBREL` → `PUBCOMP`. Both sides track the packet ID through all four steps, so the message is neither lost nor duplicated.

- **Guarantee**: the strongest MQTT offers, delivered **exactly once**
- **Cost**: three extra packets and three extra round trips versus QoS 0
- **Risk**: none to delivery correctness, but the most latency and broker state of the three levels
- **Use it for**: messages where a duplicate or a loss would actually cause harm, e.g. a billing event or a command that moves a physical actuator



# Subscribing to Topics That Don’t Exist Yet

A subscription can use **wildcards** to match a whole group of topics at once, including ones no device has published to yet.

- `+` matches exactly **one** level in the topic tree
- `#` matches **everything below** that point, any depth
- `#` is only valid as the **last** segment of a subscription

> **Tip**
>
> A publisher never uses wildcards. Only a **subscription** can use `+` or `#`; a published topic is always a concrete, fully-written name.

```
home/livingroom/temperature
home/kitchen/temperature
home/kitchen/light/status
```

- `home/#` matches **all three**
- `home/+/temperature` matches the **first two**: one level between `home` and `temperature`
- …but **not** the third: `light` adds an extra level `+` can’t cover

# Picking the Right Wildcard

## Use `+` when

- You want messages from **one specific level**, no matter what sits above or below it
- Example: every device's status, regardless of room: `home/+/status`

## Use `#` when

- You want **everything** under a prefix, whatever its shape
- Example: monitoring an entire branch of the topic tree: `home/garage/#`
- A bare `#` subscribes to **every topic on the broker**

> ⚠ **Warning**
>
> A bare `#` is useful for debugging with a tool like MQTT Explorer, but a production subscriber should scope it: an unscoped `#` on a busy broker means processing traffic meant for every other device too.

# Brokers & Clients 

Picking a broker, and the tools to poke at it

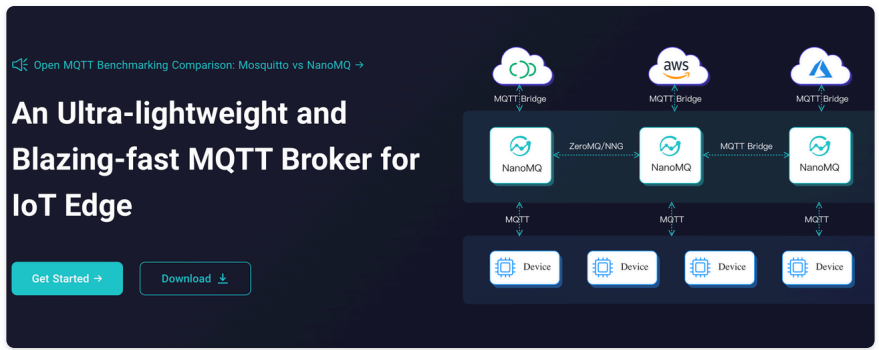
# Running Your Own Broker

**Eclipse Mosquitto** is the most widely used open-source broker: lightweight, a single config file, trivial to run in Docker.

```
docker run -d -p 1883:1883 \
  -v $(pwd)/mosquitto:/mosquitto \
  eclipse-mosquitto
```

```
# mosquitto.conf, the simplest working config
allow_anonymous true
listener 1883
```

- **EMQX** and **HiveMQ**: broker platforms with a web dashboard, clustering, and a managed cloud tier
- Both also ship a one-line Docker image for local development




NanoMQ: a lightweight broker built to run *on* the edge device, bridging up to a cloud broker.

Source: IDC – Internet de las Cosas (UPV)

# Public Test Brokers & Inspection Tools

No account needed, useful for a quick test or this workshop's demos. **Never** publish anything private to one, they're open to anyone.

- `test.mosquitto.org` (port 1883, plus a browser WebSocket demo)
- `broker.hivemq.com`
- `broker.emqx.io`
- `mqtt.eclipseprojects.io`

**MQTT Explorer** (`mqtt-explorer.com`) is the standard desktop GUI: connect to any broker, and it renders the whole topic tree live as you publish and subscribe.

> **Tip**
>
> Every major mobile platform has MQTT client apps too (search "MQTT client" on iOS or Android), handy for testing a device's publishes from a phone without writing any code.

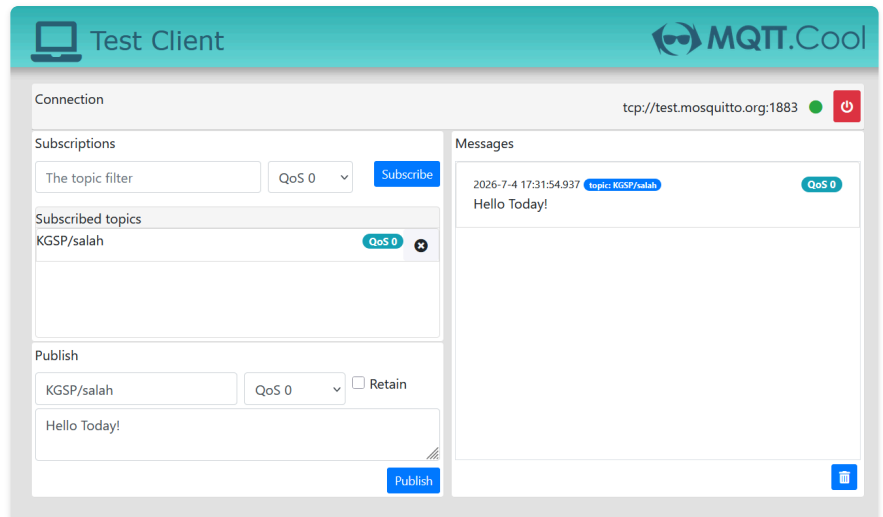
> **Note**
>
> `test.mosquitto.org` also publishes broker health under `$SYS/#`: connected client counts, message throughput, and more. Subscribing to it is a quick way to see real MQTT traffic flowing.

# Exercise: Private Chat with MQTT 💬

Open **testclient-cloud.mqtt.cool**, a free browser MQTT client, no install needed. Connect to **any broker** from the previous slide (or one of your own choosing).

- Pair up with a neighbor and agree on a **shared topic**, e.g. `chat/<your-two-names>`
- Both of you **subscribe** to that topic
- Take turns **publishing** short messages and watch them arrive on the other's screen

> **Tip**
>
> Pick a topic name unlikely to collide with anyone else in the room, these test brokers are public and shared by everyone connected to them right now.




Publishing and subscribing to `KGSP/salah` in the browser test client.

Live preview, view online at the workshop site

2026-iot.abdeljabar.net

# MQTT, from a Generic Broker to Ubidots ☁️

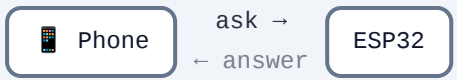
Publish sensor data to a cloud dashboard, and control an output from it, in two parts.

- Contrast a web server’s request/response model with MQTT’s publish/subscribe
- **Part 1:** talk to any broker directly with the raw PubSubClient library
- **Part 2:** move the same idea to Ubidots, a managed broker with a free dashboard
- Send real sensor readings to the cloud and receive a command back through a callback function

Key pins: DHT11 `4`, output `26` (same wiring, both parts)

# Why MQTT

**Request / response** (Projects 5 to 9)




- Your phone must be on the **same Wi-Fi**
- It has to keep **asking**, over and over
- Does not scale past the local network

**Publish / subscribe** (MQTT)


- Devices **publish** to a **broker**; others **subscribe**
- Lightweight, and works **from anywhere** with internet
- Nobody has to poll: the broker **routes and fans out**

This is the full IoT loop: DHT11 readings go **to** the cloud, and a dashboard switch comes **back** to the board.

# Broker, Topic, Publish, Subscribe

| Term | Meaning | Here |
|---|---|---|
| **Broker** | The server that routes every message | Part 1: any broker you pick. Part 2: Ubidots hosts it |
| **Publish** | Send a value to a topic | ESP32 sends temperature and humidity |
| **Subscribe** | Ask to receive a topic's values | ESP32 listens for the `led` command |
| **Topic** | The address a message is filed under | One per device and variable |

`DHT11` → `ESP32` publish → `Ubidots broker` → `Dashboard (you watch)`

`LED` ← `ESP32` ← subscribe `Ubidots broker` ← `Switch widget (you control)`

# Two Ways to Speak MQTT

**Part 1: raw PubSubClient**

```
WiFiClient espClient;
PubSubClient client(espClient);

client.setServer(MQTT_BROKER, MQTT_PORT);
client.connect(MQTT_CLIENT_ID,
               MQTT_USER, MQTT_PASS);
client.subscribe(TOPIC_LED);
client.publish(TOPIC_TEMPERATURE, payload);
```

**Part 2: Ubidots wrapper**

```
Ubidots ubidots(UBIDOTS_TOKEN);

ubidots.connectToWifi(WIFI_SSID, WIFI_PASS);
ubidots.setup();
ubidots.subscribeLastValue(DEVICE_LABEL,
                           VAR_LED);
ubidots.add(VAR_TEMPERATURE, t);
ubidots.publish(DEVICE_LABEL);
```

- Same idea underneath, still MQTT, still PubSubClient: Ubidots' library just wraps the broker address, topic naming, and Wi-Fi connection for you
- Part 1 publishes temperature and humidity as **two separate messages**; `ubidots.add()` twice then one `publish()` batches both into **one message**
- The trade: Part 1 gives you the wire protocol and free choice of broker, Part 2 trades that choice for a hosted broker, storage, and a dashboard builder

# The Cloud Talks Back: callback()

**Part 1:** text command

```
void callback(char *topic, byte *payload,
              unsigned int length) {
  // ...copy payload into message[]...
  if (strcmp(message, "ON") == 0)
    digitalWrite(LED_PIN, HIGH);
  else if (strcmp(message, "OFF") == 0)
    digitalWrite(LED_PIN, LOW);
}
```

**Part 2:** numeric switch

```
void callback(char *topic, byte *payload,
              unsigned int length) {
  // ...copy payload into value[]...
  float v = atof(value);
  digitalWrite(LED_PIN, v >= 0.5);
}
```

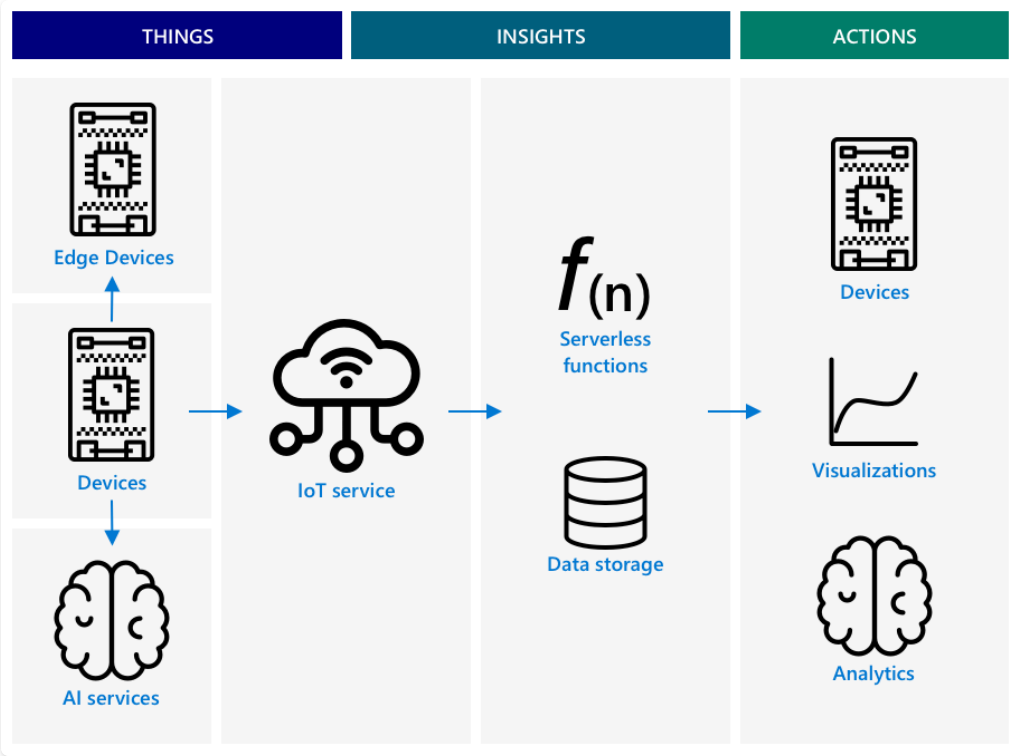
- You never call `callback()` yourself; the **library calls it** when a subscribed message arrives. That inversion is the callback pattern behind most event-driven networking.
- Same shape, different payload: a hand-typed `mosquitto_pub` command sends text ( `"ON"` ), a dashboard **switch widget** sends a number, so Part 2 parses with `atof()` instead of `strcmp()`
- `v >= 0.5` , not `v == 1` : a switch may send `1` , `1.0` , or `1.000000` , and comparing floats for equality is a bad habit

# A loop() That Heals Itself

```
if (!ubidots.connected()) {           // Wi-Fi drops, brokers restart
  ubidots.reconnect();
  ubidots.subscribeLastValue(DEVICE_LABEL, VAR_LED);  // easy to forget after a reconnect
}
if (millis() - lastPublish > PUBLISH_EVERY_MS) {
  if (!isnan(t) && !isnan(h)) {       // skip a bad reading: an honest gap beats a lie
    ubidots.add("temperature", t); ubidots.add("humidity", h);
    ubidots.publish(DEVICE_LABEL);    // both readings in one message
  }
  lastPublish = millis();
}
ubidots.loop();                       // lets incoming cloud messages be delivered
```

- The loop is now **closed**: device to cloud, and cloud to device. A device that also listens is a true IoT device.
- Network code must **self-heal**, timing must be **non-blocking** ( `delay()` would stall `ubidots.loop()` ), and the **token is a password**: keep it out of git.
- Part 1's `loop()` does the identical three jobs under PubSubClient's own names: `client.connected()` / `connectBroker()` to heal, `client.loop()` to service the connection, the same `millis()` timer to publish

# The Reference Architecture 🗺️




**Things** 🔌 Devices and edge devices sensing the world, some running AI at the edge.

**Insights** ☁️ An **IoT service** ingests the data, then **serverless functions** and **storage** turn it into meaning.

**Actions** ⚡ Commands back to devices, plus **dashboards** and **analytics** for people.

**Tip**

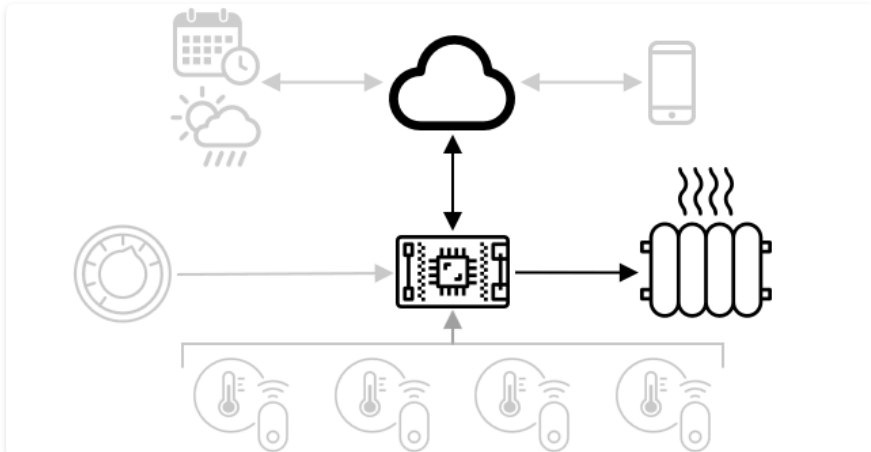
# Commands 

Controlling devices from the cloud

# Cloud → Device

A **command** is a message from the cloud telling a device to act: drive an actuator, reboot, or send extra data.

- Server checks telemetry → publishes to `/<id>/commands`
- Device subscribes and acts
- e.g. light < 300 → `{ "led_on": true }` → LED on



The cloud publishes a command; the device is already subscribed and acts on it immediately.

# When the Device is Offline

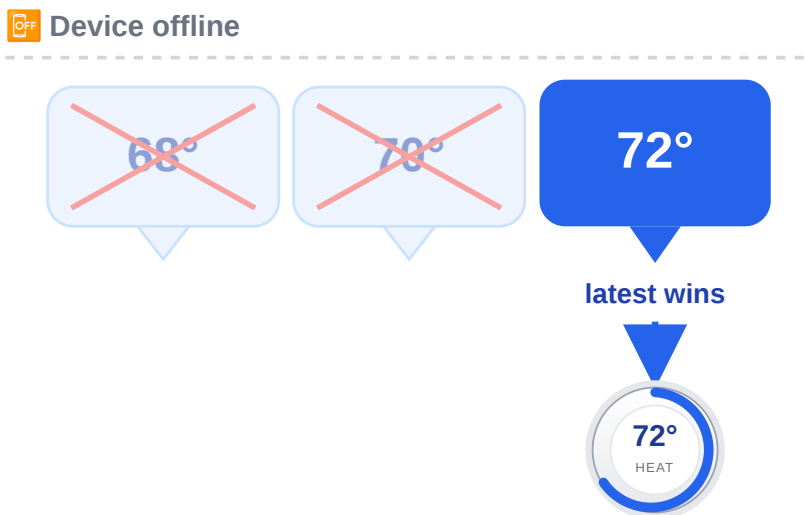
What if a command can't be delivered? **It depends on the command.**

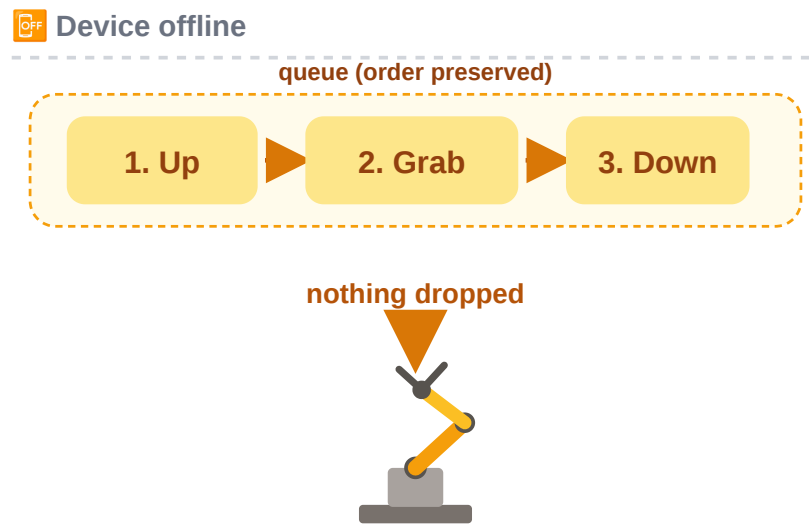
**`Latest wins → discard`**

A thermostat **setpoint**: a newer command overrides the old one, so drop the stale ones.

**`Order matters → queue`**

A **robot arm** ("up", then "grab"): commands must be **queued and replayed** in sequence.



Live preview, view online at the workshop site

2026-iot.abdeljabar.net

# Telegram Bot 🤖

Read the sensor and switch the light by chat, from anywhere.

- Use an existing chat app as the device’s whole user interface
- Poll an outside service to reach the board on any network
- Parse a text command and dispatch to the right action
- Send an unprompted message when a sensor event happens

Key pins: DHT11 `4`, output `26`, PIR `27`

Open the guide ↗

# Your ESP32 in a Chat App

Control the board and read its sensor straight from **Telegram**, from anywhere. No page to design, nothing to install.

- `/status` returns temperature, humidity, and the light state
- `/light_on` and `/light_off` switch the LED or relay, `/help` lists commands
- Set your chat id and the board **messages you** when the PIR sees motion

`You (Telegram app)` ← HTTPS → `Telegram servers` ← HTTPS → `ESP32`

Because it is just **outbound** HTTPS, this reaches your phone even on networks where hosting a web server (Projects 5 to 9) never would.

# HTTPS Needs Someone to Trust

```
WiFiClientSecure secured_client;                // TCP + TLS: the "s" in https
UniversalTelegramBot bot(BOT_TOKEN, secured_client);

// in setup():
secured_client.setCACert(TELEGRAM_CERTIFICATE_ROOT);  // trust Telegram's server
```

- A laptop ships with hundreds of **root certificates**; an ESP32 ships with **none**, so the library bundles the one Telegram chains up to and this line installs it. Skip it and every request fails, even with a valid token.
- `UniversalTelegramBot` wraps the API (turning `sendMessage()` into the right HTTPS request); **ArduinoJson** decodes the replies underneath, which is why it is a dependency.
- The **token** identifies your bot: keep it in the edit block, never public.

# Poll for Commands, Push on Motion

```
if (motion && !lastMotion && String(CHAT_ID).length() > 0)
  bot.sendMessage(CHAT_ID, "Motion detected!", "");  // push, edge-detected
lastMotion = motion;

if (millis() - lastBotCheck > BOT_CHECK_INTERVAL) {   // poll about once a second
  int n = bot.getUpdates(bot.last_message_received + 1);   // "+1": newer than last seen
  while (n) { handleNewMessages(n); n = bot.getUpdates(bot.last_message_received + 1); }
  lastBotCheck = millis();
}
```

- The board always connects **outward**, which is why it works behind a router that would block an incoming request.
- `+ 1` asks only for messages **newer than the last handled**, so the bot never replays its own history.
- `handleNewMessages()` is a **command dispatcher**: compare `text` to each command, reply to that message's `chat_id`, and answer unknowns with "send /help".

# Three Control Models

You have now built all three. The capstone is where you pick.

| Model | Reach | Strength |
|---|---|---|
| **Local web server** (P5 to 9) | Same network only | Fast, private, no account |
| **Cloud dashboard** (P11) | Anywhere | History and charts |
| **Chat bot** (P12) | Anywhere | Instant, personal, push alerts |

- The best UI is often **one people already have**: no app, no page, just `/help`
- **Outbound polling beats inbound hosting** on real-world networks
- Both directions matter: replies are **pull**, motion alerts are **push**
- Edge detection is not optional once a message **costs** something

# Security Fundamentals 🔐

What "secure" actually promises, and how it fails

# Five Things "Secure" Should Mean

🔒 **Confidentiality** Only the intended parties can read the data

✅ **Integrity** The data wasn't changed in transit

📶 **Availability** Authorized people can reach the system when they need to

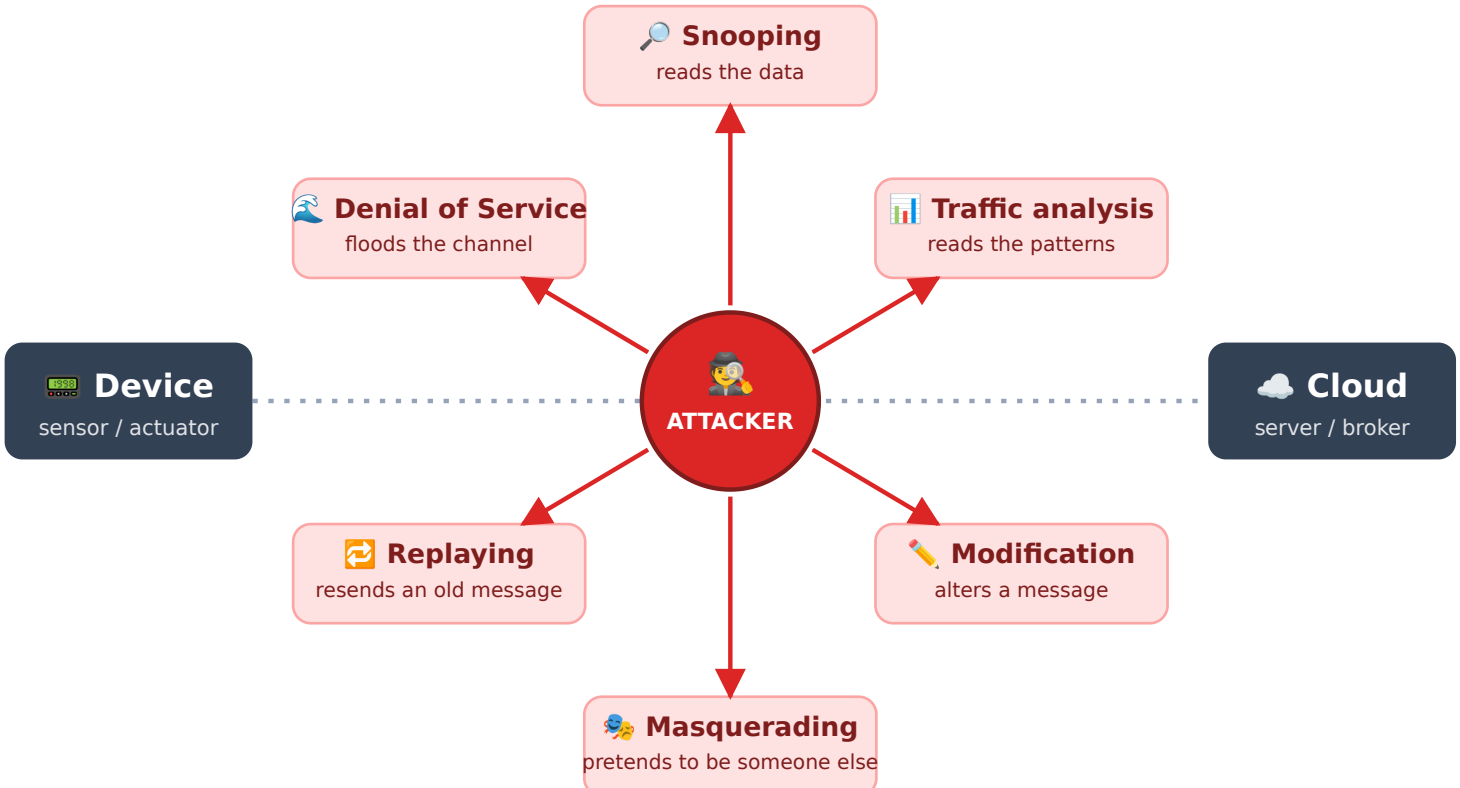
🪪 **Authentication** Whoever is connecting really is who they claim

📝 **Non-repudiation** Whoever sent something can't later deny it

> 💡 **Tip**
>
> The first three are the classic **CIA triad** (Confidentiality, Integrity, Availability). IoT adds authentication and non-repudiation front and center, since devices, not humans, are doing the talking.


Source: The Open University

# How Each Property Gets Attacked



# Why IoT Makes This Harder

Security solutions built for a data center don’t drop cleanly onto a sensor node.

## A typical server

- Grid power, no battery budget
- Megabytes to gigabytes of RAM
- Can run full antivirus and traffic analyzers

## A typical IoT node

- Battery or energy-harvested
- Kilobytes of RAM
- Heavyweight encryption drains the battery it’s trying to protect

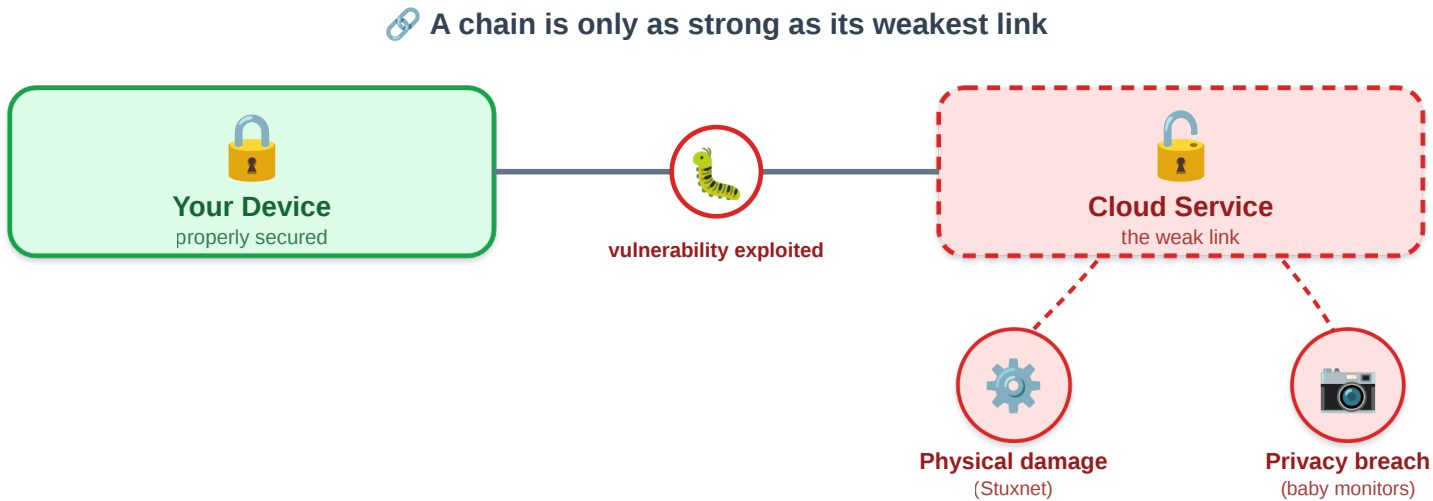
> **Note**
>
> This is why lightweight cryptography and simplified intrusion detection (watching CPU, memory, and network use for anomalies) exist as their own subfield, not a smaller version of server security.

# Security in IoT 

*"The **S** in IoT stands for Security"*

# Only as Secure as the Cloud

An IoT device connects to a cloud service, so it's **only as secure as that service**. Open connections invite malicious data and attacks, with **real-world** consequences.




**Stuxnet worm**

Manipulated **centrifuge valves** to physically destroy them.

**Hacked baby monitors**

**Weak passwords** let attackers access home cameras.

# Who Is Allowed to Connect?

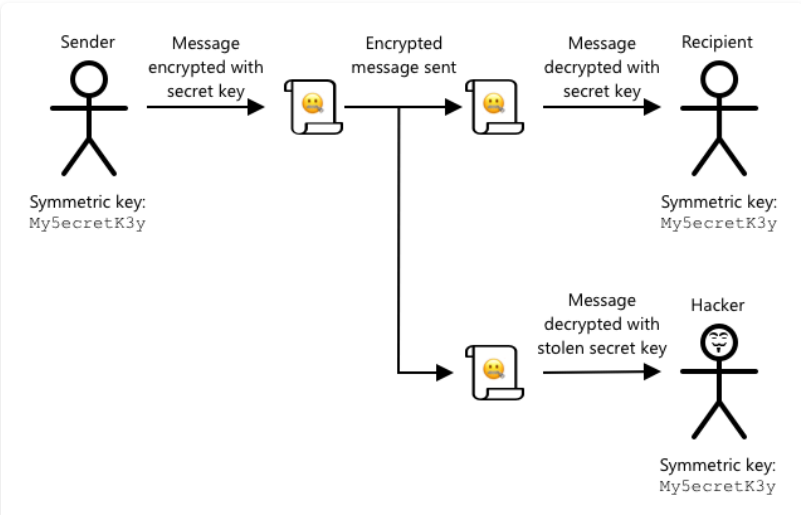

The service checks the credential first: a valid key connects, an invalid one is refused.

Every device gets its **own identity**: a secret key or certificate it presents when it connects. No valid credential, no connection.

> ⚠ **Warning**
>
> Per-device keys mean one compromised device can be revoked on its own, without locking out the whole fleet.

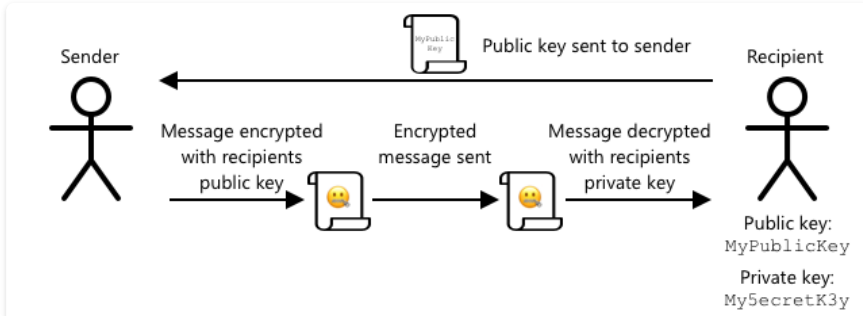
# Keeping the Message Secret

**Symmetric**: one shared key. Simple, but anyone who steals it can read everything.




The same key both locks and unlocks the message, so a stolen copy defeats it entirely.

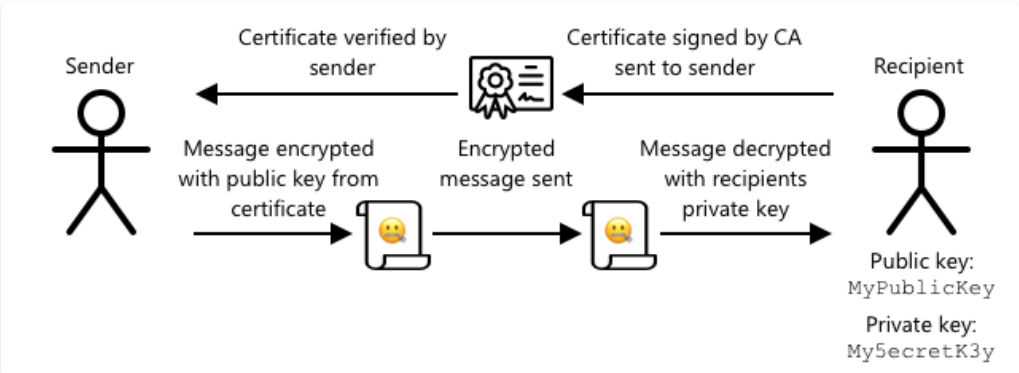
**Asymmetric**: a **public** key encrypts, only the **private** key decrypts.




Anyone can encrypt with the public key, but only the private-key holder can read it.

> **Note**
>
> Slower than symmetric, so real systems often use asymmetric keys just to agree on a symmetric one.

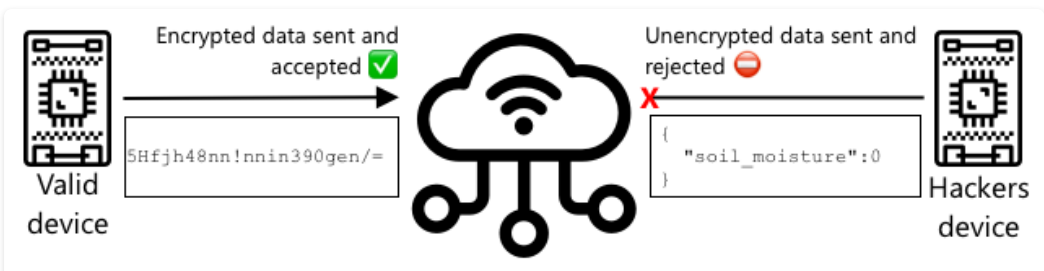
# Certificates: Proving the Key Is Theirs




The certificate authority's signature is checked before the public key inside it is trusted.

A **certificate** signed by a certificate authority proves a public key really belongs to who it claims to.

- The sender **verifies** it before trusting the key
- Devices can present certificates instead of secret keys
- The same idea behind the padlock in your browser




A certificate proves identity; encryption then keeps the data itself unreadable in transit.

Live preview, view online at the workshop site

2026-iot.abdeljabar.net

# Capstone 🎓

Design and demo an IoT device of your own choosing, starter sketch and rubric included.

- Combine a sensor, an actuator, and a network service into one device
- Scope a build to the time available
- Start from a template and extend it, the way most embedded work begins
- Debug a multi-part system by testing one piece at a time

Key pins: your choice

Open the guide ↗

# The Brief

In teams, combine the week into one small end-to-end device that does all three:

**Sense**
DHT11, PIR, LDR, pot

**Act**
LED, RGB, buzzer, relay, OLED

**Connect**
Ubidots and/or Telegram

| Idea | Sense | Act | Cloud |
|---|---|---|---|
| Smart room monitor | DHT11 + LDR | OLED readings | Ubidots charts |
| Motion security node | PIR | Buzzer | Telegram alert |
| Comfort meter | DHT11 | RGB green/yellow/red | Ubidots dashboard |

One session to build, then a short demo. A simple thing that fully works beats an ambitious thing that does not.

# Thank you!

For spending the day building the Internet of Things with us.

Questions? Ask now, or write to me any time.

Slides, labs, and resources

KGSP Summer Enrichment Program 2026
KAUST Academy, KAUST

Salah Abdeljabar
salah.abdeljabar@kaust.edu.sa